

CubeTriage

How to use it

Five steps: run it, open the queue, decide, handle the manual work, rebuild.

1 · Run the pipeline

One exported file in — a full analysis package out. ~1.5 seconds.

Inputs

a scripted .xmla database, a Tabular .bim file, an SSDT project folder (.cube + .dim), or a directory of them

Command

```
python pipeline.py <input> <outdir>
```

What you get

- *_canonical.json — the normalized model
- *_decisions.json — the work queue (stable ids)
- *_schema.sql · *_load.sql · *_validation.sql
- *_review.md · *_hitl.xlsx · *_diagram.png

Credentials

connection strings are masked before parsing — the cube's data is never read, only its metadata

2 · Open the review workspace

A local, zero-dependency web client — runs where the data lives.

Start

```
python server.py <outdir>  
-> http://localhost:8787
```

The queue

every warn flag and dropped item is a numbered decision.
Header shows progress: N of 120 decided — flags ·
assumptions · dropped. Info findings sit below as context.

Click an item

see the canonical source slice, where it lands in the star
schema, and — when verified — a proposal you can accept

Status filters

open / approved / rejected / overridden / all — plus warn
and dropped badges so priority is visible at a glance

3 · Decide — in verbs, not statuses

The button says what happens to the model, not what box is ticked.

orphan_attribute	Keep column · Drop column
calculated_measure	Leave as manual TODO · Provide SQL...
parent_child_flattened	Accept flattening · Set max depth...
scd2_incomplete	Not SCD2 — plain dimension
inactive_relationship	Keep engine's pick
not_migrated	Acknowledge — stays dropped

Custom override... always available (whitelisted canonical fields).

Comments, Save comment, Reopen — decisions are never locked.

If a flag disappears after your change, the decision is marked

stale, not deleted — the audit trail survives.

4 · Manual work — the honest backlog

Everything the engine can't produce itself, counted in the header.

Calculated measures

each row links to its decision. Provide SQL writes the expression into the canonical model — the original MDX stays for provenance

Artifact notes

guessed source tables (low column-match counts) and unresolved bridge loads — listed so verification is a checklist, not a hunt

Star schema + diagram

the List and Power BI-style diagram views show the schema the decisions produce — the model visibly grows as you work

5 · Rebuild — and done

Apply decisions -> rebuild artifacts. ~1 second per cycle.

What rebuilds

schema.sql, load.sql, validation.sql, hitl.xlsx, diagram.png
and decisions.json (merged — your decisions persist)

What changed

dropped columns are gone, provided SQL is in the load
statements, stale decisions are marked — the diff is the work

Done means

queue empty AND manual work ~0: a reproducible spec
(canonical + decisions.json) plus generated SQL that has
its own 100+-check validation harness

Prefer Excel?

fill the Beslut column in *_hitl.xlsx (ok / reject /
override field=value) and run: decisions.py import <xlsx>

In a container

server.py <outdir> --public binds 0.0.0.0 so Docker can map the
port (docker run -p 8787:8787 -v /data:/data ...) — same URL on
the host. No auth, so: trusted networks only.